
Take-home / systems For graduates & up Core stages 1–3 Stretch stages 4–5 Closed book offline docs only

SOCKETS • CONCURRENCY • REPLICATION BASICS

Build a replicated key–value store over TCP, one small step at a time.

Start with a store you can talk to over a socket. Put a proxy in front of it. Then run three copies and keep them in step. Each stage is small on its own, and we tell you which concept to look up before you get stuck — no prior distributed-systems experience assumed.

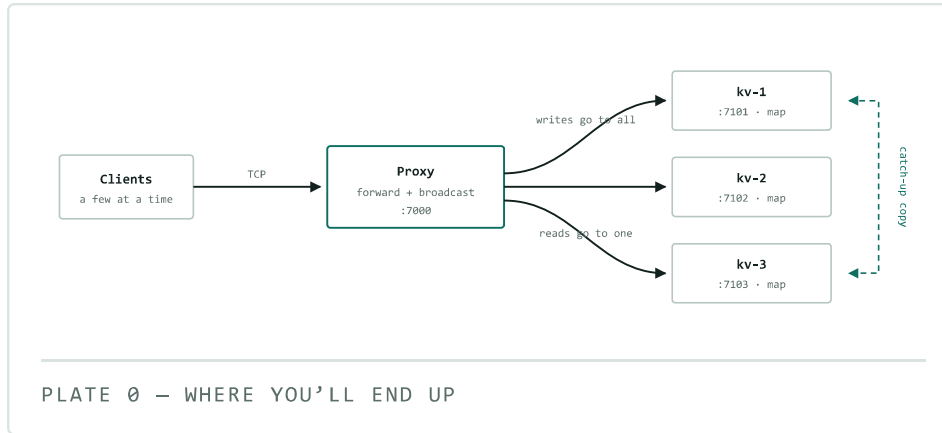
THE PICTURE AT THE END

One **proxy** that clients connect to. Behind it, **three copies** of your key–value server, each just a map in memory. A write goes to all three. A read goes to one of them. If a copy dies, the rest carry on.

That’s the whole system. No framework, no database, no cluster manager — sockets, a map, and some care about what happens when a process disappears.

HOW TO PACE YOURSELF

STAGE	STATUS
1 KV over a socket	CORE
2 Proxy in front	CORE
3 Broadcast to three	CORE



STAGE

STATUS

Stages 1–3 are the exercise. Stages 4 and 5 exist so there's somewhere to go if you finish early or find this fun — leaving them untouched costs you nothing.

GROUND RULES

LANGUAGE

Whatever you're fastest in. Standard library is plenty; an async or socket library is fine too.

REFERENCE

The full offline manuals for every language on the machine are yours — use them as much as you like. That's what they're there for.

CLOSED BOOK

No internet and no AI assistants. Everything you need is in those manuals, on this page, or available by asking us.

NOT ALLOWED

Redis, etcd,
ZooKeeper, Kafka, or a
library that does the
replication for you.

STORAGE

In memory only.
Nothing needs to
survive a restart.

TESTING

A pasted `nc` session
in your README is
perfectly good
evidence.

DELIVERABLE

Your code, plus a
README with how to
run it and what you'd
do next.

NO TRICK QUESTIONS, AND YOU CAN ASK

We're not looking for a perfect system. We're looking for how you think when a process disappears halfway through something. Where the brief doesn't say, pick whatever seems reasonable, write one line in your README about why, and carry on. And if you're stuck, come and ask — we'd much rather unblock you than watch you lose an hour. Asking costs you nothing here.

BEFORE YOU START

The concepts behind each stage

These are the ideas each stage rests on — the terms to look up in the offline manuals, or to ask us about. If a stage feels impossible, it's usually one of these rather than the code.

STAGE 1	TCP is a byte stream	message framing	accept loop	thread per connection	shared map + lock
STAGE 2	TCP proxy	port forwarding	two-way relaying	connection lifecycle	zero-byte read = EOF
STAGE 3	replication	broadcast / fan-out	health check	heartbeat	round-robin balancing
	socket timeouts				
STAGE 4	replica bootstrap	snapshot transfer	catch-up sync	anti-entropy	uptime tie-breaker
STAGE 5	read repair	majority vote	per-key versions	odd replica counts	
BONUS	quorum: $W + R > N$	eventual consistency	hinted handoff	CAP theorem	

1

CORE

~1 hour

STAGE 1

A key-value store you can talk to over a socket

Write a program that listens on a TCP port and keeps a map in memory. A client connects, sends text commands, and gets text replies. More than one client should be able to be connected at the same time — a thread per connection is a perfectly good answer.

Use the command set below so we can test everyone's the same way. Commands are one per line, ending in `\n`, so a plain `nc` session can drive it.

CLIENT SENDS	SERVER REPLIES	IF IT DOESN'T EXIST
SET <key> <value>	OK	—
GET <key>	VALUE <value>	NOT_FOUND
DEL <key>	OK	NOT_FOUND
PING	PONG	—

Keys have no spaces. A value is everything after the first space following the key. Anything you don't recognise gets `ERR bad_command`.

```
$ NC 127.0.0.1 7101

SET user:42 alice
OK
GET user:42
VALUE alice
SET user:42 alice smith # values may contain spaces
OK
GET nope
NOT_FOUND
FOO bar
ERR bad_command # and the connection stays open
```

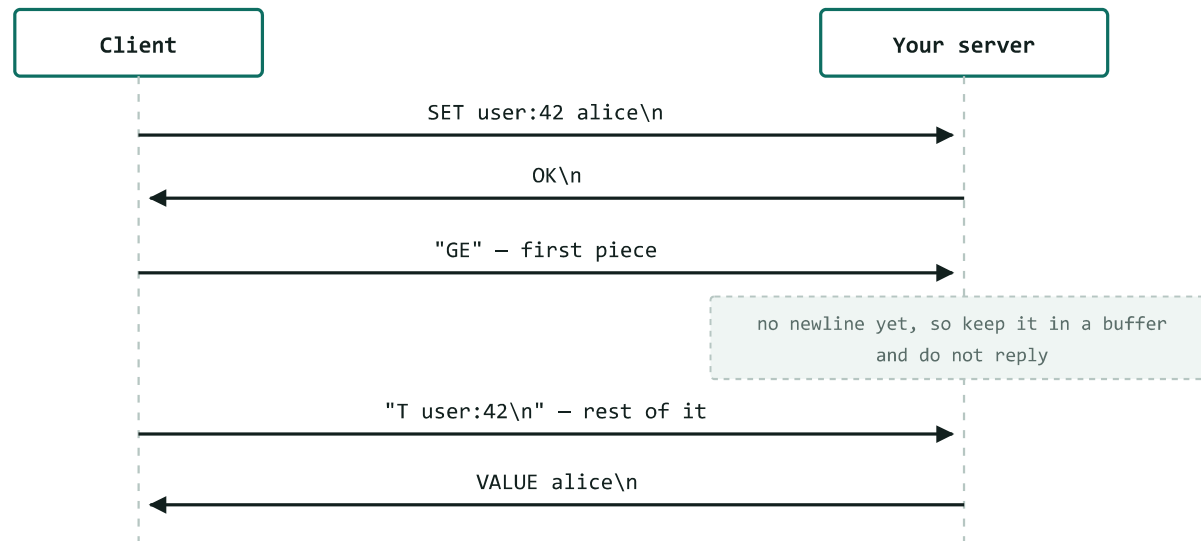


PLATE 1 — THE ONE SURPRISE: A COMMAND CAN ARRIVE IN PIECES

WHAT WE'RE LOOKING FOR

- ✓ Two clients can be connected at once without interfering with each other.
A thread per connection is a perfectly good way to get there.
- ✓ You know where one command ends and the next begins.
TCP hands you bytes, not neat lines. A single read can carry half a command, or two whole ones.
- ✓ Odd input gets an error reply instead of taking the server down.
- ✓ We can drive it with `nc` and it behaves like the table says.

HINT — THE SHAPE OF IT

- Keep a per-connection buffer. Append whatever you read, then pull off complete lines while a `\n` exists in it.
- If two threads can touch the map, guard it (a lock, or your language's concurrent map).

PSEUDOCODE — ONE VALID SHAPE, NOT THE REQUIRED ONE

```
listen on port 7101
loop:
    conn = accept()
    spawn thread(conn):
        buffer = ""
        while data = conn.read():      # empty read means the client left
            buffer += data
            while "\n" in buffer:
                line, buffer = buffer.split("\n", 1)
                conn.write(handle(line) + "\n")
```

2

CORE

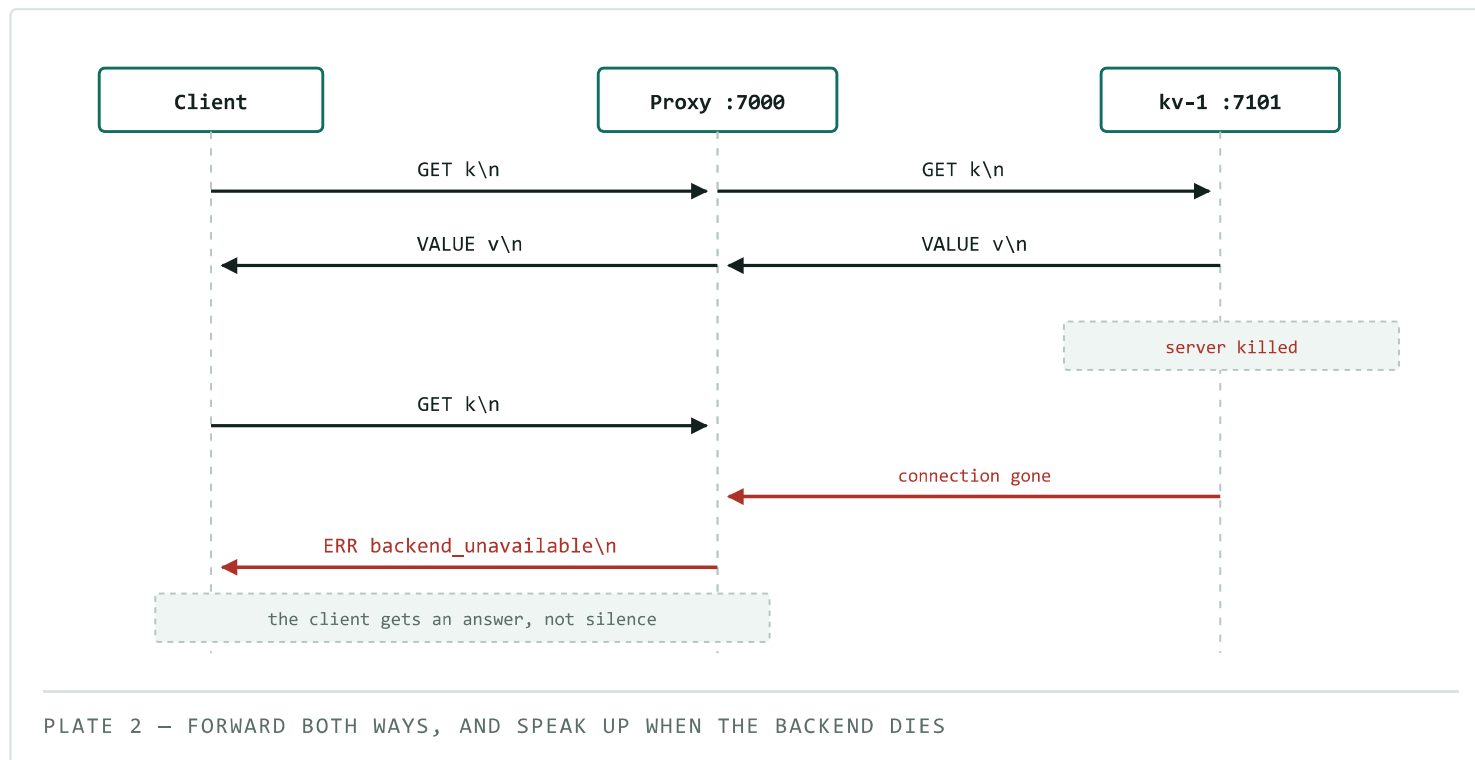
~45 min

STAGE 2

Put a proxy in the middle

Add a second program listening on port `7000`. It connects to *one* of your KV servers and passes everything through: client bytes go up, server bytes come back down. The client shouldn't be able to tell it's there.

That's all this stage is. No replication, no health checks, no decisions — you're building the pipe that the next stage puts logic into. The one thing worth getting right: when the server behind it dies, the client should hear about it rather than hang forever.



WHAT WE'RE LOOKING FOR

- ✓ Talking to the proxy gives the same results as talking to the server directly.
- ✓ Traffic flows both ways at the same time.
Reading from two sockets without one blocking the other is the part worth getting right.
- ✓ A dead backend produces an error, not a hang.
Kill the server behind the proxy and the client should still get an answer.

HINT — TWO DIRECTIONS, TWO WORKERS

- The simplest version: on each client connection, open one upstream connection and run two loops — client→server and server→client — in separate threads.
 - A read that returns zero bytes means the other side closed. That's your signal to tear the pair down.
 - Set a timeout on the upstream socket so a dead server surfaces as an error rather than a hang.
-

3

CORE

~1½ hours

STAGE 3 • THE MAIN EVENT

Broadcast every write to all three copies

Start three copies of your stage-1 server on different ports and point the proxy at all of them. Now the proxy has a job: a `SET` or `DEL` goes to **every** copy, and a `GET` goes to **one** of them.

Two rules to honour:

- A write is `OK` once the copies that are running have stored it. If **fewer than two are running**, don't accept the write — reply with an error.
- Reads keep working as long as **at least one** copy is running, and should be spread across the ones that are.

You don't need a consensus algorithm or a state machine for this. Counting how many copies are currently reachable is enough.

THE TWO RULES — CLICK A COPY TO KILL IT



SET OK — stored on all 3
GET VALUE — spread across 3

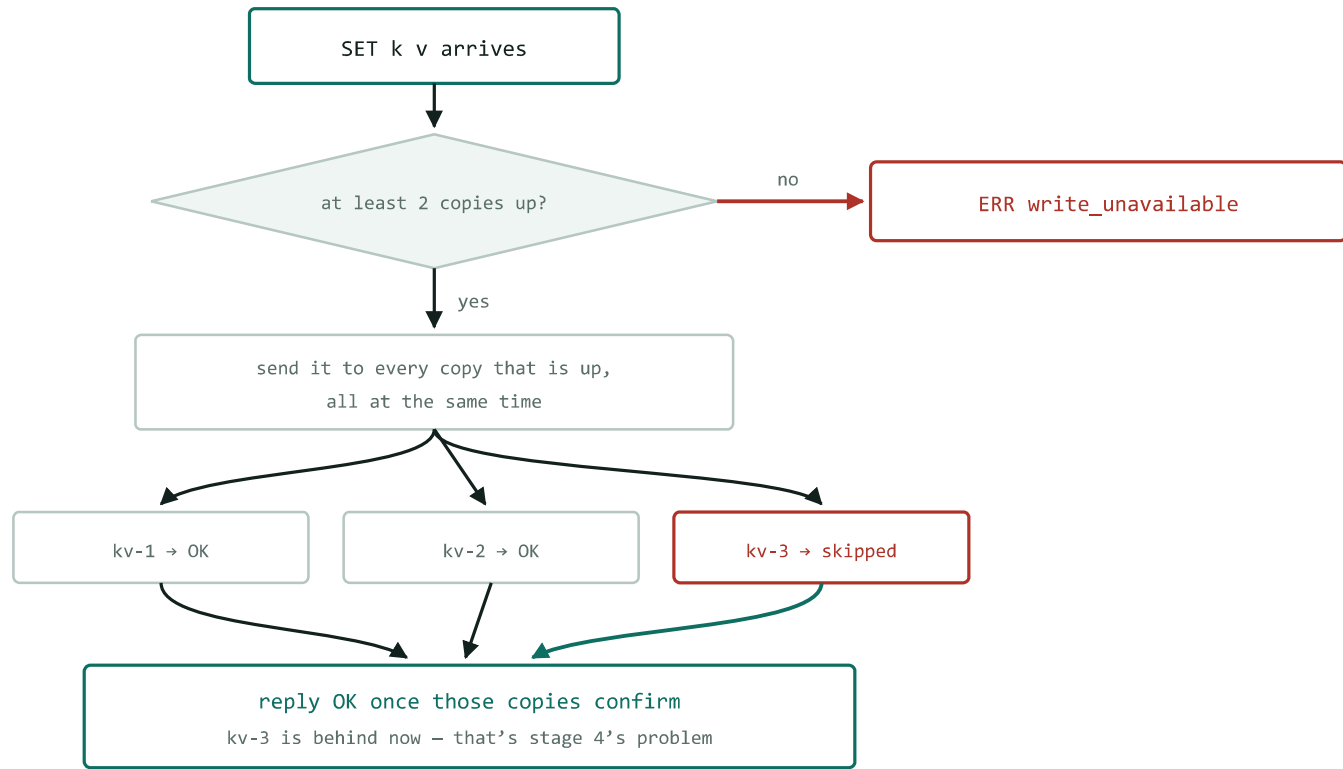


PLATE 3 – THE WRITE PATH

```
$ NC 127.0.0.1 7000 - KILLING COPIES AS WE GO

SET cart:9 2xbook
OK                                     # all three stored it
... kill kv-2 ...
SET cart:9 3xbook
OK                                     # two left, still fine
... kill kv-3 ...
SET cart:9 4xbook
ERR write_unavailable                # only one copy left, so no writes
GET cart:9
VALUE 3xbook                          # but reads still work
```

WHAT WE'RE LOOKING FOR

- ✓ A write reaches every running copy, and they end up holding the same thing.
- ✓ The copies are contacted at the same time, not one after another.
A write shouldn't take three times as long as it needs to.
- ✓ The proxy has some way of knowing a copy is down.
Pinging on a timer, or simply noticing a failed write, are both fine. Tell us which you chose.
- ✓ Reads don't all land on the same copy when several are available.
- ✓ With two copies dead, writes are refused but reads keep answering.
The two errors should be tellable apart by the client.

HINT — WHERE PEOPLE LOSE TIME

- Keep one small structure in the proxy: for each copy, its address and whether you currently believe it's up. Everything in this stage is a read of that structure.
- Send the write to each copy from its own thread and collect the replies, rather than looping through them in order.
- Always put a timeout on those writes. A copy that's hung is worse than one that's dead, because nothing tells you.
- Reads getting spread out can be as simple as a counter that increments, modulo the number of copies that are up.

DELIBERATELY NOT ASKED

Replying as soon as *two of the three* have confirmed, without waiting for the third — that's a *quorum*, and it's the classic next step. It's in the bonus list, not here. For this stage, waiting for the copies you sent to is exactly right.

4

STRETCH

~1 hour

STAGE 4 • OPTIONAL

Bring a restarted copy back up to date

A copy you killed in stage 3 comes back with an empty map, so it's missing everything written while it was gone. Before it starts answering reads again, have it copy the current contents from the **oldest running copy** — the one that's been up the longest.

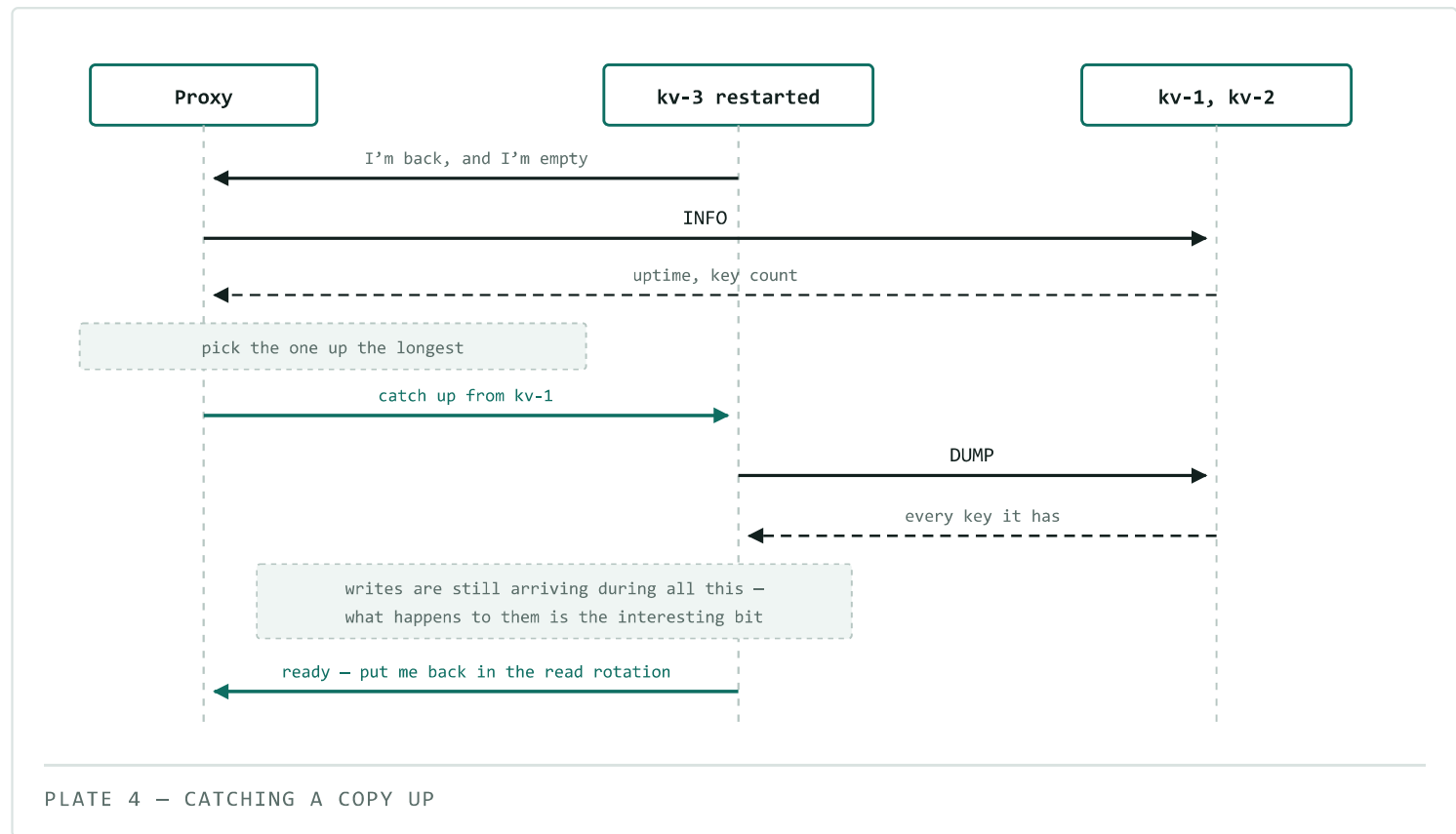
You'll want two more commands to make this possible. Shape them however you like; this is the idea:

INFO

Tells you how long this copy has been up and how many keys it holds — that's how you find the oldest one.

DUMP

Sends back everything it has, so a fresh copy can load it in one go.



WHAT WE'RE LOOKING FOR

- ✓ Given two running copies, you can say which is older and always pick the same one.
- ✓ A copy that's still filling up doesn't serve reads.
An empty answer is worse than no answer.
- ✓ Some way to tell whether the catch-up actually worked, rather than assuming it did.

- ✓ You've thought about the writes that arrive while the copying is going on.
A working solution is great. A clear account of the problem and how you'd handle it is worth nearly as much.

HINT — AND A QUESTION WORTH ANSWERING

- Record a start timestamp when a server boots; uptime is just now minus that. Ties broken by name keeps it deterministic.
- For “did it work”, a cheap trick is a checksum over the sorted contents. Same number on both copies means they agree.
- In your README: is “oldest” really the best copy to learn from? When would “the one with the most keys” be a better pick?

5

STRETCH

~45 min

STAGE 5 • OPTIONAL

When one copy disagrees, trust the majority

A copy can be running, answering quickly, and still be *wrong* — a bug, a bad merge, a stray write. So for this stage, ask **all three** copies for the value instead of one, and go with the answer at least two of them give.

To make a copy wrong on purpose, add something like `CORRUPT <key> <value>` that quietly changes a value on that one server. Then read through the proxy and see if it notices.

WHAT THE THREE COPIES SAY

WHAT THE PROXY SHOULD DO

All three agree

Return the value.

Two agree, one differs

Return what the two say, and fix the odd one out.

All three differ

Return an error — there's no majority to trust.

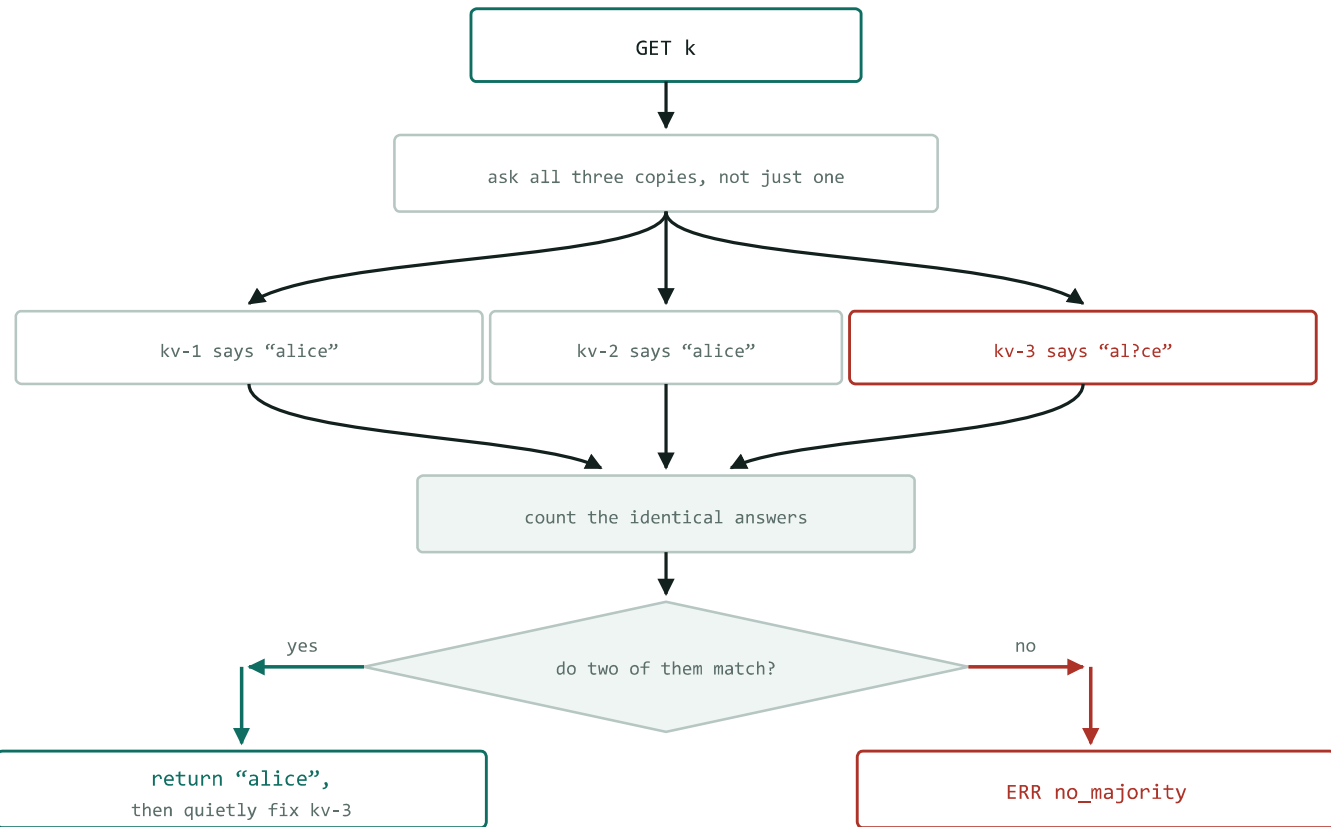


PLATE 5 – MAJORITY READ, THEN REPAIR

```
$ NC 127.0.0.1 7000 - AFTER CORRUPTING ONE COPY

SET user:42 alice
OK
... CORRUPT user:42 al?ce sent straight to kv-3 ...
GET user:42
VALUE alice # kv-1 and kv-2 outvote kv-3
... a moment later, asking kv-3 directly ...
GET user:42
VALUE alice # it got fixed
```

WHAT WE'RE LOOKING FOR

- ✓ The proxy notices that the three answers aren't the same.
- ✓ It returns the value two copies agree on, not the first one that replied.
- ✓ It makes some attempt to fix the copy that was wrong.
- ✓ Something visible marks that it happened.
A log line or a counter is enough for a human to spot it.

HINT — AND THE WRINKLE WORTH NAMING

- Group the three replies by value and take the group with two or more in it. That's the whole algorithm.
- Careful: a copy can disagree because it's *corrupt*, or because it's *behind*. Those deserve opposite treatment, and you can't tell them apart from the value alone. A version number per key is how real systems break the tie — implement it if you have time, mention it if you don't.

→ Also worth a line in your README: this triples your read traffic. Would you leave it on all the time?

If you want to go further

Only worth starting once stages 1–3 work. Pick one, not all.

- Quorum writes** — reply `OK` as soon as two of three have stored the value, and let the third catch up on its own. This is the real-world version of stage 3; be ready to explain what you traded away.
- Version numbers** — give every key a version that goes up on each write, and use it to settle disagreements between copies.
- Queue writes for a dead copy** — hold on to what a down copy missed and replay it when it returns, instead of dumping everything.
- A chaos script** — randomly kill, delay and corrupt copies for a minute, and check nothing broke.

How we'll read your submission

There's no checklist being ticked off line by line, and no hidden test suite. Four things matter to us, in this order:

WHAT TO SEND US

- ☐ Your code, in a repo or a zip.
- ☐ How to start it — a script, a Makefile, or just the commands to run in order.
- ☐ A short README: what works, what doesn't, and how far you got.
- ☐ A pasted terminal session showing it working.
- ☐ Anything you decided for yourself where the brief was vague.
- ☐ One paragraph on what you'd do next with another day.

QUICK ANSWERS

Do I need to finish all five stages?

Get as far as you can in the time you have. Stages 1–3 first; 4 and 5 only if there's room left.

What if I get stuck for an hour?

- Does it work** for the cases described, and can you show us?
- What happens when something dies** — do clients get answers, and does the system come back?
- Can we follow the code** and does it do what your README claims?
- How you talk about it** — the trade-offs you chose, and where you know it's weak.

Three stages done properly, with honest notes about the rest, reads far better than five stages half-finished. We'd rather see your reasoning than your stamina.

Don't — ask us. Note down where you got stuck and why, then come and ask. Both the note and the question count in your favour.

Can I search the web or use an AI assistant?

No, neither. The offline language manuals on the machine are there for exactly this, and we're happy to be asked.

Threads, async, or processes?

Any of them. Threads are the easiest to reason about here.

Does data need to survive a restart?

No. Everything lives in memory.

Do I need tests, auth, or TLS?

None of them. A terminal session is enough evidence.

Can I change the command names?

Yes, as long as your README says what they are.